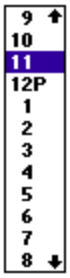
Lists



Lists provide a fast way for you to create a potentially large set of options from which the user can choose and to have that set of options scroll automatically. If you have limited space on your form and usability would not suffer due to the additional tap necessary, then consider displaying the list with a pop-up trigger.

There are three different ways to provide a list with its contents. This article contains recipes for all three:

- Pre-Determined content
- Dynamic content
- Custom-drawn content

Lists With Pre-Determined Content

The easiest way to create a list is to have the contents of the list pre-determined at build time. To do this, create the list in Constructor as follows:

1. Create a new list object, name it "Day" for this example. Place it wherever you want on your form.
2. Set "Visible Items" to 7. The height of the list is based on the number of items and the chosen font.
3. Add the strings for the list items. (Add items by clicking "List Items" and choosing "New Item Text," or typing command/control K.) For this example, add the days of the week: Monday, Tuesday, and so on, one day in each item.
4. Adjust the width of the list to accomodate the widest string. 52 pixels is enough, assuming you don't need to scroll and are using the standard font.

Now you need to add code to your `frmOpenEvent` to initialize the current selection of the list. (Often you'll do this sort of work in a function called `MainFormInit` or something similar.) Use code like this:

```
ListType *lstP;
FormType* frmP = FrmGetActiveForm();

lstP = (ListType *) FrmGetObjectPtr(frmP, FrmGetObjectIndex(frmP,
    MainDayList));
LstSetSelection(lstP, 3);  // selects the fourth item in the list

//note that LstSetSelection(lstP, -1); can be used
//to unselect all items
```

Your list is now functional, but to be useful you need to add code to find out when the user changes the selection or at least to read the selected value at some point. Here is a sample handler for the `lstSelectEvent`, which is sent when the user chooses an item from the list.

```
case lstSelectEvent:
     if (eventP->data.lstSelect.listID == MainDayList) {
        Int16 theSelection;
        Char *theText;

        theSelection = eventP->data.lstSelect.selection;
        theText = LstGetSelectionText(eventP->data.lstSelect.pList,
                theSelection);
       }
```

If you only need to retrieve the selected value at a later time, instead of in response to `lstSelectEvent`, use `LstGetSelection`.

Lists With Dynamic Content

Often the choices in a list are determined at runtime. When the form is opening, you can provide a list of strings to the list object with `LstSetListChoices`. You also must add code to free the list of choices when the form closes.

To make a dynamic list, first create the list resource in Constructor:

1. Create a new list object and name it "Years".
2. Set "Visible Items" and the coordinates of the list as you wish. We will have 8 items in our list, so make it shorter if you want to see the automatic scrolling.
3. Don't create any list items; that will be done at runtime.

In your `frmOpenEvent` handler, in addition to setting the default choice, you to create the list's contents. Use code like the following:

```
   lstP = (ListType *) FrmGetObjectPtr(frmP, FrmGetObjectIndex(frmP,
        MainYearsList));
    FillListWithYears(lstP);
    LstSetSelection(lstP, 0);
```

Here is the function that allocates the strings for the list and fills them with data. Notice that it uses two global variables to keep track of the memory it allocates:

```
   char **gYearListChoices=0;
    Int16 gYearListNumItems;

    static void FillListWithYears(ListType *lst)
    {
       const numYears = 8;
       Char **itemList;
       DateTimeType dt;
       Int16 year;

       itemList = (Char **) MemPtrNew(numYears * sizeof(Char *));
       gYearListChoices = itemList;  // save so we can free it later
       gYearListNumItems = numYears;

       TimSecondsToDateTime(TimGetSeconds(), &dt);
       for (year=0; year <numYears; year++) {
```

```
         // just enough for a year string
         itemList[year] = (char *) MemPtrNew(6);
         StrPrintF(itemList[year], "%d", year+dt.year);
      }
      LstSetListChoices(lst, itemList, numYears);
   }
```

As with the case of lists with pre-determined content, you will probably have a handler for `lstSelectEvent`, and/or you will get the selection with `LstGetSelection`. But you now also need to free the memory that was allocated by `FillListWithYears`, as follows:

```
case frmCloseEvent:
      if (gYearListChoices) {
         UInt16 i;
         for (i=0; i<gYearListNumItems; i++)
            MemPtrFree((MemPtr) gYearListChoices[i]);
         MemPtrFree((MemPtr) gYearListChoices);
         gYearListChoices=0;
      }
      break; //remember to leave handled as false
```

The complexity resulting from having to remember to free the list choices often makes this technique worse than having a custom drawing routine for the list. However, this way works much better than having a custom drawing routine if the list of choices is slow to generate or must be generated sequentially.

Custom-Drawn Lists

When the items in a list must be generated at runtime and can be quickly generated in any order, use a callback function to draw the contents of the list instead of providing a list of strings.

First, generate the list resource in Constructor:

1. Create the list object on your form and name it "Times" for this example.
2. Make sure the list is at least 48 pixels wide so that the automatically added scroll arrows won't overlap the longest text item (in our case, "12:00 pm")
3. The list's height is determined by the font and the number of visible items; set them as you choose.
4. Don't create any list items; that will be done at runtime.

In your `frmOpenEvent`, you need to specify the draw function as well as the number of items in the list:

```
  ListType *lstP;

   lstP = (ListType *) FrmGetObjectPtr(frmP, FrmGetObjectIndex(frmP,
       MainTimesList));
   LstSetDrawFunction(lstP, TimesDrawCallback);
   LstSetListChoices(lstP, NULL, 24); // display all 24 hours
```

The list manager calls the draw function for each line in the list that needs to be drawn. The draw function is passed the coordinates and item number of that one list item. The font and colors have already been set appropriately, so it is a simple function.

```
static void TimesDrawCallback(Int16 itemNum, RectangleType *bounds,
     Char **itemsText)
 {
    char buffer[timeStringLength];

    // the itemNum is the hour; convert it to ascii.
    TimeToAscii(itemNum, 0, tfColonAMPM, buffer);

    // the font and colors have already been set; just draw.
    WinDrawChars(buffer, StrLen(buffer), bounds->topLeft.x,
             bounds->topLeft.y);
 }
```

For this example, the item number is all the information we need, because our list simply contains a list of the hours from midnight until 11pm. Other applications may need other information such as a record index or a pointer to the item. In those cases, it may be easiest to have the list choices array be an array of the record indexes or whatever data is necessary. If you have set a drawing function, then the list object never looks at the data inside the list choices array that you pass to `LstSetListChoices`, so you can have any information you want in that `itemsText` array.

Note: Do not call `LstGetSelectionText` if you set `itemsText` to be something other than the list items or if you never set `itemsText` at all. the `LstGetSelectionText` function bases its return value on the `itemsText` array.

Because calling `LstGetSelectionText` is often not possible when you provide a custom drawing function, it's often convenient to have a single function that copies the text for a given item number into a buffer in the same way `LstGetSelectionText` does. The above example uses `TimeToAscii` for this purpose.

One of the strong benefits of having a draw function is that you do not need to worry about keeping track of memory that would have to be allocated for the list's contents. However, the drawing callback function will be called often. In addition to being called to draw each item, it is also called (with different foreground and background colors) to highlight or un-highlight items as the user taps them, so it needs to be fast. If it isn't fast enough, consider creating the list of strings beforehand. Another option is to have your callback function generate each item as necessary but store them in `itemsText` for fast access in case they are needed again.